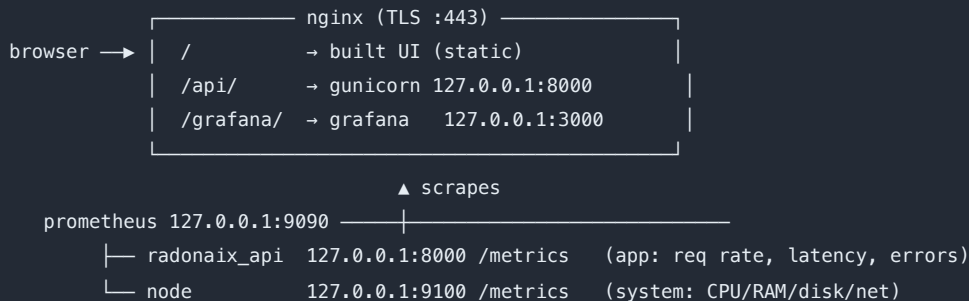


RADONAIX — Monitoring (Prometheus + Grafana)

System + application monitoring for the bare-VM (systemd) deployment. Everything runs **locally** on the app server and is reached through the **existing nginx HTTPS origin** — no new public ports.



The app already exposes Prometheus metrics at `/metrics` (root, **not** under `/api`):

`http_requests_total{method,path,status}` and `http_request_duration_seconds{method,path}`

(`app/core/middleware.py` , `app/main.py`). Prometheus scrapes gunicorn directly on `127.0.0.1:8000` — `/metrics` is not exposed through nginx.

The artifacts referenced below live under `deploy/` : `deploy/prometheus/prometheus.yml` , `deploy/grafana/**` , `deploy/systemd/{node_exporter,prometheus}.service` .

1. node_exporter (system metrics)

```
# Download the binary (pick the current version from prometheus.io/download).
VER=1.8.2
cd /tmp && curl -fsSL0
https://github.com/prometheus/node_exporter/releases/download/v${VER}/node_exporter-${VER}.linux-amd64.tar.gz
tar xzf node_exporter-${VER}.linux-amd64.tar.gz
sudo mkdir -p /opt/monitoring/node_exporter
sudo cp node_exporter-${VER}.linux-amd64/node_exporter /opt/monitoring/node_exporter/
sudo useradd -rs /bin/false node_exporter

# Service (binds to 127.0.0.1:9100)
sudo cp /opt/radonaix/backend/deploy/systemd/node_exporter.service /etc/systemd/system/
sudo systemctl daemon-reload && sudo systemctl enable --now node_exporter
curl -s http://127.0.0.1:9100/metrics | head    # sanity check
```

2. Prometheus (collection)

```
VER=2.55.1
cd /tmp && curl -fsSL0
https://github.com/prometheus/prometheus/releases/download/v${VER}/prometheus-${VER}.linux-amd64.tar.gz
tar xzf prometheus-${VER}.linux-amd64.tar.gz
sudo mkdir -p /opt/monitoring/prometheus /etc/prometheus /var/lib/prometheus/data
sudo cp prometheus-${VER}.linux-amd64/prometheus /opt/monitoring/prometheus/
sudo cp /opt/radonaix/backend/deploy/prometheus/prometheus.yml /etc/prometheus/
# Scrape targets live in their own files (file_sd) – copy the whole dir. The
# `files:` paths in prometheus.yml are absolute (/etc/prometheus/targets/*.json).
sudo cp -r /opt/radonaix/backend/deploy/prometheus/targets /etc/prometheus/
sudo useradd -rs /bin/false prometheus
sudo chown -R prometheus:prometheus /var/lib/prometheus

# Validate the config, then start
/opt/monitoring/prometheus/promtool check config /etc/prometheus/prometheus.yml
sudo cp /opt/radonaix/backend/deploy/systemd/prometheus.service /etc/systemd/system/
sudo systemctl daemon-reload && sudo systemctl enable --now prometheus
curl -s http://127.0.0.1:9090/-/healthy          # "Prometheus Server is Healthy."
```

After ~30s, check **Targets** (<http://127.0.0.1:9090/targets> via SSH tunnel, or trust the Grafana dashboards): `radonaix_api` and `node` should both be **UP**.

3. Grafana (dashboards)

```
# Install from the Grafana RPM repo (RHEL/Rocky 9)
sudo tee /etc/yum.repos.d/grafana.repo >/dev/null <<'EOF'
[grafana]
name=grafana
baseurl=https://rpm.grafana.com
repo_gpgcheck=1
enabled=1
gpgcheck=1
gpgkey=https://rpm.grafana.com/gpg.key
EOF
sudo dnf install -y grafana

# Config (see deploy/grafana/grafana.ini): merge it, or set in /etc/grafana/grafana.ini:
# [server] root_url = https://<host>/grafana/ ; serve_from_sub_path = true
# [security] allow_embedding = true ; so the UI can iframe dashboards
# [auth.anonymous] enabled = true ; org_role = Viewer ; embed renders without login
# Set the admin password WITHOUT hardcoding it in the file:
sudo mkdir -p /etc/systemd/system/grafana-server.service.d
sudo tee /etc/systemd/system/grafana-server.service.d/override.conf >/dev/null <<'EOF'
[Service]
Environment=GF_SECURITY_ADMIN_PASSWORD=CHANGE_ME_STRONG_PASSWORD
EOF

# Provision the datasource + dashboards
sudo cp -r /opt/radonaix/backend/deploy/grafana/provisioning/* /etc/grafana/provisioning/
sudo mkdir -p /var/lib/grafana/dashboards
sudo cp /opt/radonaix/backend/deploy/grafana/dashboards/*.json /var/lib/grafana/dashboards/
sudo chown -R grafana:grafana /etc/grafana/provisioning /var/lib/grafana/dashboards

sudo systemctl daemon-reload && sudo systemctl enable --now grafana-server
```

4. nginx — expose Grafana on the app origin

The `/grafana/` block is already in `deploy/nginx/radonaix.conf`. Make sure the deployed conf includes it, then:

```
sudo nginx -t && sudo systemctl reload nginx
```

Open `https://<host>/grafana/` → log in (`admin` / the password you set) → the **RADONAIX** folder has *System (node_exporter)* and *API health* dashboards.

5. Firewall / security

- Keep **9090, 9100, 3000 closed** to the outside — all three bind to `127.0.0.1` and are reached only locally (Grafana via nginx). `bash`
they should NOT appear as open in: `sudo firewall-cmd --list-ports`
- Anonymous **Viewer** access is enabled so the embedded iframe renders without a login prompt. This makes the dashboards read-only-viewable to anyone who can reach the app origin — acceptable for an internal deployment behind TLS. Editing/admin still needs a real Grafana login (admin password via the systemd override). To require login even for viewing, set `[auth.anonymous] enabled = false` and the embed will show Grafana's sign-in inside the frame.
- nginx sets `X-Frame-Options: SAMEORIGIN` for `/grafana/` (overriding the global `DENY`) so the same-origin UI can iframe it. Grafana's `allow_embedding = true` removes its own frame block.

- `/metrics` stays internal — nginx never proxies it (the root path falls through to the SPA).

6. UI (embedded)

The UI **System Monitoring** screen (`/monitoring` , gated by the `settings` RBAC permission) **embeds** the Grafana dashboards in an iframe (kiosk mode, tabbed System / API). It defaults to the relative `/grafana` path (same origin); set `VITE_GRAFANA_URL` at build time if Grafana lives elsewhere. A header button still opens the full dashboard in Grafana.

Local dev (UI on :8080, Grafana on :3000 — cross-origin), run Grafana with embedding on:

```
GF_PATHS_PROVISIONING=/tmp/graf/provisioning \
GF_PATHS_DATA=/tmp/grafana-data GF_PATHS_LOGS=/tmp/grafana-logs \
GF_SECURITY_ADMIN_PASSWORD=admin \
GF_SECURITY_ALLOW_EMBEDDING=true \
GF_AUTH_ANONYMOUS_ENABLED=true GF_AUTH_ANONYMOUS_ORG_ROLE=Viewer \
grafana server --homepath "$(brew --prefix grafana)/share/grafana"
```

and set `VITE_GRAFANA_URL=http://localhost:3000` in the UI `.env` .

7. Email alerts (Grafana alerting)

Threshold alerts email `manojbm@platum-ai.co.in` + `kalyanm@platum-ai.co.in` (from `platum07@gmail.com`) when the host crosses a threshold. All provisioned from the repo (`deploy/grafana/provisioning/alerting/` + the `[smtp]` block in `deploy/grafana/grafana.ini`):

- **Rules** (`rules.yaml`): High CPU >85% (5m), High memory >85% (5m), High disk on `/` >85% (5m), monitoring target down (3m). Tune thresholds/ `for` there.
- **Contact point** (`contactpoints.yaml`): `radonaix-ops` (the two recipients).
- **Policy** (`policies.yaml`): route everything to `radonaix-ops` (repeat every 4h while firing).

Deploy:

```
sudo cp -r /opt/radonaix/backend/deploy/grafana/provisioning/alerting /etc/grafana/provisioning/
sudo chown -R grafana:grafana /etc/grafana/provisioning/alerting
# merge the [smtp] block into /etc/grafana/grafana.ini (host/user/from), then set the
# Gmail APP PASSWORD (needs 2FA on platum07@gmail.com) via the systemd override:
sudo tee -a /etc/systemd/system/grafana-server.service.d/override.conf >/dev/null <<'EOF'
[Service]
Environment=GF_SMTP_PASSWORD=YOUR_GMAIL_APP_PASSWORD
EOF
sudo systemctl daemon-reload && sudo systemctl restart grafana-server
```

Verify: Grafana → **Alerting** → **Contact points** → `radonaix-ops` → **Test** → both inboxes get a mail; **Alerting** → **Alert rules** shows the 4 rules (Normal). Force one (e.g. `stress / dd`) to confirm a real alert email fires after the `for` window. `journalctl -u grafana-server | grep -i smtp` for delivery errors.

Gmail: `platum07@gmail.com` must have 2FA + a 16-char **App Password** (plain passwords are rejected for SMTP). The app password lives **ONLY** in `GF_SMTP_PASSWORD` — never commit it.

8. Postgres connections (postgres_exporter)

Exposes connection metrics (vs `max_connections`) shown in the **API health** dashboard (Postgres panels) so the alert can warn before "too many clients". One exporter sees the whole instance.

```
# 1) binary
VER=0.15.0
cd /tmp && curl -fSL -o pgexp.tar.gz \
  https://github.com/prometheus-
community/postgres_exporter/releases/download/v${VER}/postgres_exporter-${VER}.linux-amd64.tar.gz
tar xzf pgexp.tar.gz
sudo mkdir -p /opt/monitoring/postgres_exporter
sudo cp postgres_exporter-${VER}.linux-amd64/postgres_exporter /opt/monitoring/postgres_exporter/
sudo useradd -rs /bin/false postgres_exporter 2>/dev/null || true
sudo chown -R postgres_exporter:postgres_exporter /opt/monitoring/postgres_exporter

# 2) DB connection string (creds) in an EnvironmentFile - NOT committed. Prefer a
#   least-privilege monitoring role (GRANT pg_monitor TO <user>).
sudo mkdir -p /etc/monitoring
echo 'DATA_SOURCE_NAME=postgres://postgres:postgres@10.200.37.142:5432/postgres?sslmode=disable' \
  | sudo tee /etc/monitoring/postgres_exporter.env >/dev/null
sudo chmod 600 /etc/monitoring/postgres_exporter.env

# 3) custom query for total/active/idle (the unit loads it via --extend.query-path)
sudo cp /opt/radonaix/backend/deploy/monitoring/postgres_exporter_queries.yaml /etc/monitoring/

# 4) service (binds 127.0.0.1:9187)
sudo cp /opt/radonaix/backend/deploy/systemd/postgres_exporter.service /etc/systemd/system/
sudo systemctl daemon-reload && sudo systemctl enable --now postgres_exporter
curl -s http://127.0.0.1:9187/metrics | grep -cE "pg_stat_database_numbackends|radonaix_pg_connections_active" #
> 0

# 5) Prometheus already has the `postgres` job (deploy/prometheus/prometheus.yml). Its
#   targets live in /etc/prometheus/targets/postgres.json - edits there are hot-reloaded.
#   Only re-copy + restart when prometheus.yml itself changes:
sudo cp /opt/radonaix/backend/deploy/prometheus/prometheus.yml /etc/prometheus/
sudo cp -r /opt/radonaix/backend/deploy/prometheus/targets /etc/prometheus/
sudo systemctl restart prometheus
```

The Postgres KPIs render in the **API health** dashboard (used % with total/active on hover, and connections by database), and the **"Postgres connections near limit (>80%)"** alert emails ops before the limit is hit. Keep **9187 firewall-closed** (localhost only).

If you previously deployed the standalone dashboard, remove it: `sudo rm -f /var/lib/grafana/dashboards/radonaix-postgres.json && sudo systemctl restart grafana-server`

Verify (end to end)

```
systemctl is-active node_exporter prometheus grafana-server # all "active"
curl -s http://127.0.0.1:9090/-/healthy
curl -s http://127.0.0.1:8000/metrics | grep -c http_requests_total # > 0
```

Then load `https://<host>/grafana/` and confirm both dashboards show live data, and that the UI sidebar shows **System Monitoring** for an admin and hides it for a role without `settings` view.

9. The monitored fleet — adding a server

One **Prometheus + Grafana** (on the master report server) pulls from **every** other host. You never install Prometheus/Grafana again — you install **exporters** on the new box and add one **scrape target**. One pane of glass.

Targets are NOT in `prometheus.yml`. Every job uses file-based service discovery (`file_sd_configs`) reading `deploy/prometheus/targets/*.json`. Prometheus **watches those files and hot-reloads** — no `systemctl reload`, no restart.

The fleet: role → exporters

Server(s)	server id	role	Exporters
App 1–4 (AIR / SDP / SVC / Exception)	<code>app-air</code> , <code>app-sdp</code> , <code>app-svc</code> , <code>app-exc</code>	<code>app</code>	node <code>:9100</code> + postgres <code>:9187</code> (Airflow metadata DB)
ClickHouse master / slave	<code>ch-master</code> , <code>ch-slave</code>	<code>db</code>	node <code>:9100</code> + ClickHouse native <code>:9363</code>
Postgres master / slave	<code>pg-master</code> , <code>pg-slave</code>	<code>db</code>	node <code>:9100</code> + postgres <code>:9187</code>
Report 1 / 2 (UI, Superset, Grafana, Prometheus, app DB)	<code>rpt-master</code> , <code>rpt-slave</code>	<code>report</code>	node <code>:9100</code> + postgres <code>:9187</code> + API <code>/metrics</code>
File archive · DB archive	<code>arch-file</code> , <code>arch-db</code>	<code>archive</code>	node <code>:9100</code>

Every target carries labels: `server` (id) · `display` (UI label) · `role` · `tier` (`master` / `slave`) · `feed` (`air` / `sdp` / `svc` / `all`, app servers only).

Adding a server — two steps

1) On the new box (installs the right exporters for the role, relabels the binaries for SELinux, and firewalls the ports to the Prometheus host only):

```
sudo ./deploy/monitoring/install_exporters.sh <app|db-postgres|db-clickhouse|report|archive> \  
--bind-ip <this-host-ip> --prom-host <prometheus-host-ip>
```

The script runs `restorecon` on each binary. Skipping that is what causes systemd `status=203/EXEC` — the exact failure we hit installing `postgres_exporter`.

2) On the Prometheus host — append one object to the matching targets file:

```
// deploy/prometheus/targets/nodes.json (always)  
{ "targets": ["10.200.36.70:9100"],  
  "labels": { "service":"host", "server":"pg-slave",  
              "display":"DB 4 · Postgres Slave (70)", "role":"db", "tier":"slave" } }
```

plus `postgres.json` / `clickhouse.json` if the role has them. Prometheus picks it up **within 30s**. Nothing else to do:

- **Grafana System dashboard** — its `$server` variable is `label_values(up{job="node"}, server)`.
- **Alerts** — every rule groups `by (server)`; the email names the host via `{{ $labels.server }}`.
- **The UI server selector** — `GET /api/monitoring/servers` reads the live `server` / `display` labels straight from Prometheus, so the button appears with **no rebuild and no env var**. (This is why the list can never drift out of sync with what's actually scraped.)

Order matters: install the exporters *before* adding the target, or `radonaix-target-down` fires while Prometheus can't reach the new host.

ClickHouse metrics

`install_exporters.sh db-clickhouse` writes `/etc/clickhouse-server/config.d/prometheus.xml` (endpoint `:9363`), then **restart ClickHouse**. This exposes `MemoryTracking` vs the server memory cap — the signal that would have predicted the recon-report OOM — plus merges, query errors and replication delay.

HA: two report servers, one alerting voice

`rpt-master` and `rpt-slave` both run Prometheus + Grafana and both scrape the fleet (data redundancy). Only `rpt-master` has **alerting enabled** — provision `rpt-slave` identically but leave its alert rules **paused**, and enable them on failover. Otherwise every incident emails twice.

Still to do (Phase 3, once those exporters are live)

Role dashboards + alert rules for **ClickHouse** (memory vs cap, replication delay) and **Postgres replication** (lag). They are deliberately *not* pre-written: the metric names depend on the ClickHouse / postgres_exporter versions actually deployed, and an unverified Grafana provisioning file fails Grafana at startup. Add them when `ch-master` / `pg-slave` first report.

Phase 3 UI design (agreed) — server selector + view tabs

The System Monitoring screen gets a **two-level control**: a server selector above the existing view tabs.

```
Server: [ Server 1 · SDP (142) ] [ Server 2 · AIR (133) ]
View:   [ System ] [ API health ]
```

- The buttons rewrite the iframe URL with a Grafana template variable, e.g. `/grafana/d/<uid>?var-server=sdp-142&kiosk&theme=light&refresh=30s` (and `air-133`).
- **System** dashboard uses a `server` template variable (`label_values(up{job="node"}, server)`); every panel filters `node_...{server="$server"}`. One dashboard serves both hosts — no per-server duplicate. `air-133` appears in the dropdown automatically once its exporters are live.
- **API health is 142-only**. Our FastAPI app (the `radonaix_api` job / `/metrics`) runs **only on .142**; .133 runs AIR dags (Airflow), not our backend. So when **Server 2 (133)** is selected, the **API health tab is hidden/disabled** and only **System** is shown. (If the app is ever deployed on .133 too, add a `radonaix_api` scrape target for it and the tab becomes available there with no other change.)

Production fleet (target topology)

Production is ~8 hosts. The **same** central Prometheus + Grafana monitors all of them; each host runs only the relevant exporter(s). Nothing about the model changes — you only add scrape targets and `server` label values.

Role	Host(s)	Exporters	Notes
App — AIR dags	<code>app-air-01</code>	node	Airflow host; no <code>radonaix_api</code>
App — SDP dags	<code>app-sdp-01</code>	node	Airflow host
App — MSC dags	<code>app-msc-01</code>	node	Airflow host
DB — Postgres (master)	<code>db-pg-master</code>	node, postgres	HA primary
DB — Postgres (slave)	<code>db-pg-slave</code>	node, postgres	HA replica; watch replication lag
DB — ClickHouse (master)	<code>db-ch-master</code>	node, (clickhouse)	CH exporter optional/later
DB — ClickHouse (slave)	<code>db-ch-slave</code>	node, (clickhouse)	replica
Reporting	<code>report-01</code>	node, <code>radonaix_api</code> , postgres (app DB)	runs our UI + backend + Prometheus + Grafana

Label schema (set in `static_configs.labels` per target so dashboards/alerts can slice the fleet):

- `server` — unique host id (the selector value, e.g. `app-air-01`, `db-pg-master`). **Required**.
- `role` — `app` | `db` | `reporting` (group dashboards/alert routing by tier).

- `stream` — `air` | `sdp` | `msc` (app hosts) — ties a host to its pipeline.
- `engine` — `postgres` | `clickhouse` (db hosts).
- `ha` — `master` | `slave` (db hosts) — distinguishes HA pairs in panels/alerts.

Example target with the full schema:

```
- job_name: postgres
  static_configs:
    - targets: ["db-pg-master:9187"]
      labels: { service: postgres, server: db-pg-master, role: db, engine: postgres, ha: master }
    - targets: ["db-pg-slave:9187"]
      labels: { service: postgres, server: db-pg-slave, role: db, engine: postgres, ha: slave }
```

What this enables without rework:

- The System dashboard `$server` dropdown lists every host; an optional `$role` variable can filter the dropdown to one tier (`label_values(up{role="$role"}, server)`).
- Per-server alerts already fan out by `server`; HA pairs are distinguishable by `ha`.
- The UI Server selector grows by adding entries to `SERVERS` in `monitoring.tsx` (the `id` must match the `server` label); `API_HOST` stays the reporting host.

Deferred for the fleet (not now): ClickHouse exporter (CH connection/query metrics), Postgres **replication-lag** alert on the slave, and per-role alert routing (e.g. DB alerts → DBA contact point). Capture these when the production hosts exist.

Implemented now (2 servers)

- Grafana System dashboard: `server` variable + per-server panel filters.
- Alert rules: per-server grouping + server named in notifications.
- UI: Server selector (Server 1 · SDP 142 / Server 2 · AIR 133) + System/API view tabs, API hidden for non-app servers.
- Prometheus: `server` labels live for `.142`; `.133` targets pre-staged (commented). Remaining to activate `.133`: Phase 2 server-side steps above.

Out of scope (later)

Datastore exporters (postgres/clickhouse/nginx), Alertmanager + notifications, anonymous-viewer iframe embedding, and long-term remote-write storage.